

Rapport TER

Master MIASHS parcours IC
2025-2026

Projet JADE

Justice Algorithmes Des Élections

Tuteur de stage au LIG : Maximin Coavoux

LIG Grenoble : Bâtiment IMAG, 700 Av. Centrale, 38401 Saint-Martin-d'Hères

Tuteur de TER à l'UGA : Damien Pellier

UGA : 1251 Rue des Universités, 38400 Saint-Martin-d'Hères

Mainguet Louane
Toumpanos Antony

Table des matières

Rapport TER.....	1
Projet JADE.....	1
Table des matières.....	2
Présentation projet JADE.....	3
Présentation des objectifs.....	3
Organisation du travail.....	4
Notre travail.....	4
Récupération des données.....	4
Modèle.....	6
Site web pour l'annotation manuelle.....	8
Bonne pratique.....	8
Les problématiques.....	9
Amélioration.....	9
Conclusion.....	10

Présentation projet JADE

Le projet de recherche interdisciplinaire JADE “Justice algorithmique des élections” , porté par l’Université Grenoble-Alpes, a pour objet d’utiliser des méthodes de statistiques appliquées à un objet juridique pour lequel elles n’ont jamais été utilisées en France, le contentieux des élections politiques, dans le but d’aider à en comprendre les mécanismes et la rationalité.

Le sujet était divisé en deux TER :

- Un sujet portant sur le développement de la base de données relationnelle du projet
- Un sujet sur le développement d’un modèle de traitement automatique des langues (le sujet sur lequel nous avons travaillé)

Nous travaillons sur un corpus de décisions de justice dans lequel certains segments (appelés 'considérants') sont annotés à l'aide d'une des étiquettes suivantes, correspondant à un type d'argument juridique :

- 1. procédure
- 2. opération préalable au scrutin
- 3. candidature
- 4. campagne : propagande
- 5. campagne : pressions et manœuvres
- 6. financement
- 7. operation

Notre travail consiste à fournir un classifieur basé sur un modèle de langue préentraîné capable de prédire le label de décisions de justice qui ne sont pas encore classifiées, afin de faciliter le travail des juristes chargés de cette catégorisation.

Pour ce faire, nous nous sommes appuyés sur une base de documents déjà existants. En effet, plusieurs mois auparavant, un premier travail avait été réalisé par un ingénieur, Louis Maritaud. Son modèle produisait déjà des résultats préliminaires, qui nous ont été présentés au début du TER.

Présentation des objectifs

Notre objectif principal au début du projet était donc de récupérer le modèle de Louis Maritaud, de le relancer, puis d’en améliorer les résultats en modifiant les paramètres afin de classer de nouvelles décisions de justice.

Cependant, nous avons rencontré diverses problématiques tout au long de notre TER. Cela a modifié les objectifs du travail. Nous avons finalement dû recréer un

modèle à partir de zéro afin de classifier les données, mais aussi nous occuper de la mise en forme des données utilisées par le modèle, ainsi que la création d'un site web pour aider à la correction des prédictions de notre modèle.

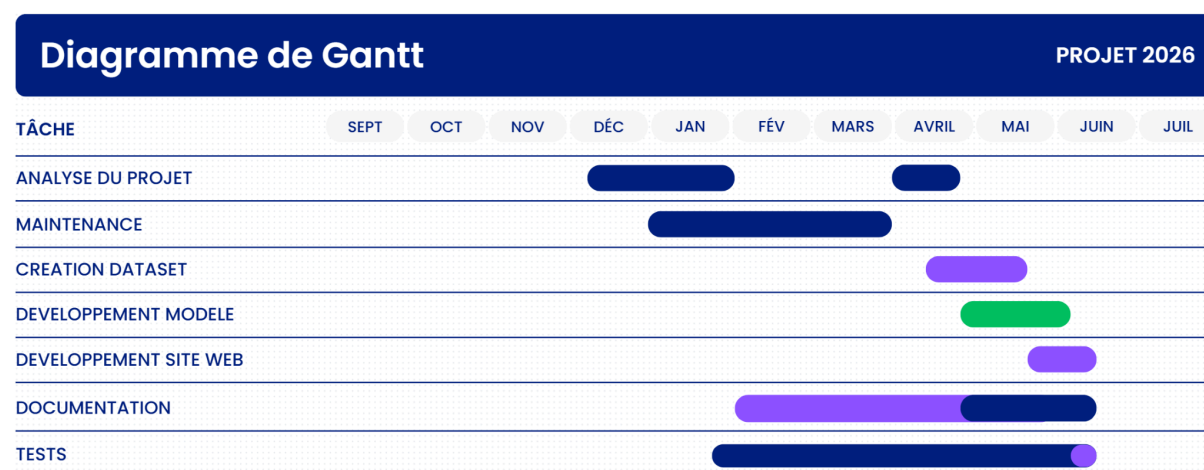
Organisation du travail

Pendant la première phase du TER, réalisée à raison d'une journée par semaine, nous avons décidé de mettre en place une réunion avec notre tuteur toutes les deux semaines. Cette réunion, assimilable à une *sprint review*, avait pour objectif de passer en revue nos avancées au cours du sprint et de définir les tâches à intégrer dans le sprint suivant.

Lors de la seconde phase, effectuée à temps complet, nous avons opté pour de courts *daily meetings* chaque matin. Une *sprint review* était organisée une fois par semaine avec nos deux maîtres de stage, Maximin et Fang.

En complément, trois réunions générales ont été tenues avec l'ensemble des membres du projet JADE : data scientists, juristes, gestionnaires de base de données et développeurs. Une première réunion s'est déroulée en décembre, suivie d'une seconde début mai, puis d'une troisième fin mai. Enfin, la réunion finale a eu lieu début juin.

Ci-dessous se trouve un diagramme de Gantt présentant en vert les tâches réalisées par Antony, en violet celles réalisées par Louane, et en bleu les tâches effectuées ensemble.



1. diagramme de gantt

Notre travail

Récupération des données

La création du jeu de données se fait via deux fichiers aux objectifs distincts. Dans un premier temps, `Before_training.py` permet de parcourir tous les fichiers présents dans le

dossier AN (Assemblée Nationale) et d'en récupérer le contenu. Pour cela, il effectue une extraction du contenu de la balise <contenu> du XML traité, puis effectue une étape de data cleaning. L'objectif de cette étape est de formater les données d'une seule manière afin de faciliter le travail de machine learning. Cette première étape génère un json nommé output.json qui est organisé de la manière suivante :

```
[{
  "id": "CONSTEXT000017665826",
  "considerants": [{
    "numero": 1,
    "text": "Considérant qu'en soulevant un moyen tiré de ce que les procès-verbaux
de recensement des votes de certains bureaux font apparaître un excédent dans le nombre
des enveloppes et bulletins sans enveloppe trouvés dans l'urne par rapport au nombre des
émargements, ...",
    "label": []
  },{}]}
], ...]
```

2. arborescence de output.json et considerants_avec_label.json

On retrouve une liste contenant plusieurs dictionnaires. Chaque dictionnaire possède l'identifiant du fichier XML traité ainsi que la liste des considérants qu'il contient, chacun ayant son propre numéro. Un même fichier pouvant comporter entre un et vingt considérants, il est nécessaire de conserver l'identifiant du considérant dans la variable `numero` (de type `int`) . La variable `text` (de type `string`) correspond au contenu textuel du considérant, c'est-à-dire la partie située après son numéro. Enfin, une variable `label` (de type `list` vide) est ajoutée afin de recueillir le futur label attribué par le modèle. Ce fichier JSON contient environ 44 000 lignes, correspondant au contenu de 15 908 fichiers XML.

Maintenant que la structure du JSON est clairement définie, il est nécessaire d'y intégrer les labels corrigés attribués aux considérants. Ces labels corrigés proviennent d'un fichier Excel réalisé par Louis Maritaud en 2025. À la suite d'une itération de son modèle, une phase d'annotation manuelle a été effectuée dans laquelle une partie des considérants s'est vu attribuer un label corrigé.

Voici quelques exemples issus de la colonne *Annotation alternative proposée* :

(3. Candidature, 7. Opérations, 1. Irrecevabilité, 1. Procédure, 1. procédure, 2. Opérations préalables au scrutin, 2. Préalable scrutin, ...)

De plus, certaines variations se sont glissées dans les catégories Campagne et Opération. La catégorie Campagne est en réalité divisée en deux sous-catégories : 4. campagne et 5. campagne. Cependant, certains annotateurs ont nommé la catégorie 5. campagne : pressions et manœuvres de différentes manières (5. Campagne : pressions et manœuvres, 5. Campagnes : pressions, 5. Campagne, etc.), tandis que 4. campagne n'apparaît que sous une seule forme.

C'est pour cette raison qu'une phase de *data cleaning* a été effectuée, comprenant un formatage des données : suppression des accents, des pluriels et des espaces inutiles. L'objectif est de standardiser au maximum les labels afin de simplifier le travail d'entraînement du modèle.

Après cette étape, le label correspondant à la colonne *Annotation alternative proposée* a été ajouté dans le fichier `output.json`, en conservant exactement la même structure que l'exemple ci-dessous.

```
"label": ["7. operation prealable au scrutin"]
```

Afin de vérifier la bonne mise en forme des labels, nous nous sommes aidés d'un script Python nommé `Comptage.py` :

```
Nombre total de labels attribués : 1117
4. campagne : propagande : 383
7. operation : 317
3. candidature : 193
6. financement : 160
None : 32
5. campagne : pressions et manœuvres : 15
1. procedure : 13
2. operation prealable au scrutin : 3
1. irrecevabilite : 1
```

3. répartition des labels dans le dataset

Modèle

Nous avons donc en parallèle travaillé sur la création puis l'entraînement d'un modèle à partir de ces données.

Il y avait environ 1 100 exemples annotés que nous avons divisés en trois parties :

- 70% dans le corpus de train
- 15% dans le corpus de dev
- 15% dans le corpus de test

```
--- train.json (750 éléments) ---
4. campagne : propagande : 268
7. operation : 222
3. candidature : 135
6. financement : 112
5. campagne : pressions et manœuvres : 10
2. operation prealable au scrutin : 3
--- dev.json (160 éléments) ---
4. campagne : propagande : 57
7. operation : 47
3. candidature : 29
6. financement : 24
5. campagne : pressions et manœuvres : 3
--- test.json (161 éléments) ---
4. campagne : propagande : 58
7. operation : 48
3. candidature : 29
6. financement : 24
5. campagne : pressions et manœuvres : 2
```

4. image répartition de chaque label dans chaque corpus

On peut voir qu'il n'y a aucun exemple annoté par un label nul, un label "irrecevabilité" ou "procédure" conformément aux consignes. Comme il n'y avait que 3 exemples avec le label

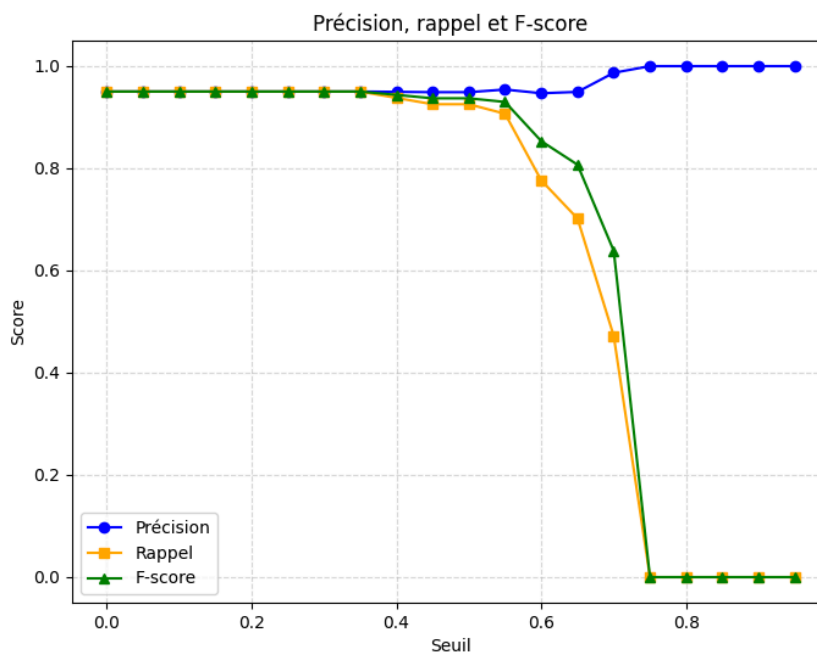
n°2, nous n'avons pas inclus cette classe dans l'évaluation et avons placé ces 3 exemples dans le corpus d'entraînement.

Pour le choix du modèle, nous avons opté pour un fine-tuning (plutôt que d'entraîner un modèle from scratch) de CamemBERT , un modèle de langue Transformer de type encodeur pré-entraîné sur un large corpus de textes en français. Ce choix a été fait pour la langue de notre corpus à classifier, et a été conservé pour les résultats qu'il a donné.

Le nombre d'époques a été défini à 3 puisque c'est pour 3 époques que l'évaluation du modèle donnait les meilleures performances sur le corpus de développement, après avoir testé jusqu'à 6 époques.

Ce modèle a donc ensuite été testé sur le corpus de test, et nous avons illustré ses performances par des indicateurs chiffrés comme la précision et le rappel : Avec ces indicateurs, notre modèle semble être fiable à plus de 91% (les précisions sont bonnes dans plus de 91% des cas).

Nous avons aussi testé ces indicateurs (ainsi que le F-score, qui est la moyenne harmonique de la précision et du rappel) pour différents seuils de confiance de prédictions du modèle, ce qui a donné le graphique suivant :



5. graphique des performance du modèle (précision, rappel, F-score)

On voit que le seuil change surtout le compromis entre couverture et fiabilité. Quand le seuil augmente, on garde moins de prédictions, et donc la précision augmente mais le rappel chute. Par ailleurs, il est très performant; avant la chute de rappel, les 3 métriques dépassent 0,9.

En utilisant ce modèle, nous avons donc pu prédire un label pour chacun des exemples qui n'en possédaient pas à l'intérieur de `considerant_avec_label.json`, fichier créé par le code `recup_label.py`.

Site web pour l'annotation manuelle

Ce site web Python fonctionne via un serveur Flask permet de charger un fichier Json local sur le site et de le parcourir. Le site permet aux annotateurs d'avoir une interface plus simple que le fichier excel anciennement utilisé. A l'origine, les considérants ainsi que le chemin absolu et l'annotation du modèle de Louis Maritaud, ingénieur temporairement responsable du code en 2025, étaient exportés dans un excel. Cela permettait ensuite aux annotateurs de rentrer leurs corrections ainsi qu'une explication si nécessaire.

Le site utilise les logs de Flask afin d'informer le développeur en cas de mode debug. Nous avons utilisé une norme de nommage pour les commentaires afin de faciliter la maintenance du site par de futurs intervenants.

6. image site web d'annotation des considérants après entraînement du model

Afin que n'importe qui possédant un ordinateur windows puisse lancer le site web, y compris les annotateurs (spécialiste du droit), nous avons réalisé un fichier exécutable. De plus, une documentation simple et illustrée permet une compréhension rapide de l'utilisation de l'interface git.

Bonne pratique

@brief pour décrire le contexte et l'objectif de la fonction, **@param** et **@return** pour détailler les paramètres d'entrée, leurs types, leur utilité, ainsi que les valeurs retournées.

Lors de la reprise du code de Louis Maritaud, nous avons d'abord forké son projet afin de pouvoir effectuer nos tests sans perturber le versionning du dépôt principal. Cependant, en raison de l'absence de documentation et de la difficulté de maintenir l'ancien code, nous avons finalement abandonné ce fork.

Nous avons ensuite créé notre propre projet sur le GitHub partagé avec Damien Pellier. Ce nouveau dépôt nous a permis de construire notre propre dataset et d'entraîner un nouveau modèle.

À l'arrivée de notre nouveau co-maître de stage, Fang, nous sommes retournés sur le gricad du projet. Nous avons décidé de structurer le développement en 4 branches distinctes :

`main`, `stage`, `louane` et `antony`. Chacun dispose ainsi de sa branche de travail personnelle. Lorsqu'une fonctionnalité est finalisée, un merge est effectué vers la branche `stage`. Enfin, après validation, un merge de `stage` vers `main` vient intégrer la fonctionnalité dans la version stable du projet.

Les problématiques

La mission initiale du TER consistait à reprendre un modèle de TALN développé par Louis Maritaud et à en assurer la maintenance afin de l'améliorer. Nous avons été confrontés à de nombreuses reprises à des fichiers ou dossiers manquants, pourtant indispensables au lancement du code. Une partie importante des fichiers et des explications se trouvait sur un espace partagé auquel nous n'avons eu accès que fin avril.

De plus, ce sujet nécessitant plusieurs expertises, toute question liée au métier devait être posée par mail ou faire l'objet d'un rendez-vous planifié à l'avance.

De nombreux problèmes ont également été relevés en début d'année concernant les librairies utilisées par Louis Maritaud. En effet, l'absence de documentation et de versionnage des dépendances et du langage utilisé a rendu le code impossible à exécuter.

Enfin, l'utilisation d'un modèle nommé Jargon, reposant sur la librairie fairseq, elle-même incompatible avec Windows, a fortement limité nos possibilités de maintenir le code. En effet, fairseq dépend de nombreuses librairies obsolètes, également utilisées par le modèle, et qui ne sont plus compatibles avec les versions récentes de Python et les librairies nécessaires à l'exécution du modèle.

Amélioration

Ce stage nous a également permis d'identifier plusieurs améliorations qu'il reste à mettre en place.

Au niveau du modèle, il serait pertinent d'analyser les résultats de ses prédictions en les faisant corriger par des juristes afin de pouvoir l'adapter en conséquence. De plus, un plus grand nombre de métriques d'évaluation et des indicateurs divisés par labels seraient une amélioration de la visualisation des performances du modèle.

Pour le site d'annotation, il serait notamment intéressant d'ajouter une liste déroulante permettant de sélectionner un fichier XML parmi l'ensemble des documents disponibles. Après sélection, une seconde liste déroulante pourrait afficher les considérants contenus dans ce fichier. L'objectif est de permettre aux annotateurs de commencer où ils le souhaitent et ainsi de mieux se répartir le travail.

De plus, il serait utile de permettre la reprise de l'annotation au même endroit dans le fichier JSON après un clic sur le bouton `Save`. Deux solutions sont envisageables :

- Extraire l'index du fichier XML et du considérant à partir du nom du fichier sauvegardé, puis reprendre l'annotation à cet endroit.

- Parcourir le JSON généré lors du clic sur **Save**, retrouver le dernier élément annoté, puis identifier son index dans le fichier global des considérants. L'annotation pourrait alors reprendre directement à partir des éléments non encore traités.

Seul un exécutable pour Windows est créé excluant les utilisateurs Linux et Mac. Il faut donc créer des exécutables pour Linux et Mac.

L'hébergement du site web d'annotation sur les serveurs du LIG ou de l'université pourrait se révéler être une solution plus simple et accessible que la création et la mise à jour de différents exécutables.

Conclusion

Au cours de notre TER, nous avons réalisé plusieurs scripts permettant de mettre en forme le dataset nécessaire à la création du modèle. Nous avons conçu un classifieur permettant de prédire le type d'argument juridique associé à un considérant. Enfin, nous avons développé un site web permettant d'annoter les considérants à partir du JSON généré par le modèle, et d'en produire un nouveau afin de poursuivre l'entraînement. Ce TER nous a permis de mettre en pratique les connaissances théoriques acquises en statistique ainsi qu'en IA par la création d'un nouveau modèle. Les cours de gestion de projet nous ont permis de mieux structurer notre compréhension du projet ainsi que de se pencher sur une meilleure documentation du code. En tant qu'informaticien, rendre notre travail accessible et maintenable représente une compétence que nous avons pu mettre à profit dans le cadre du projet JADE.